

MINISTRY OF EDUCATION



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

STUDY BUDDY

SEMESTER PROJECT

Authors: **Capac Teodora**
 Oarga Ioan-Sergiu
 Pal Robert
 Plăcintar Ioana-Ariana

Scientific coordinator: **Prof. Ing. Ioan Valentin Sita**

2026

Table of contents

1	INTRODUCTION.....	2
1.1	PROJECT CONTEXT	2
1.2	OBJECTIVES	2
1.3	SPECIFICATIONS.....	2
2	BIBLIOGRAPHIC RESEARCH	3
3	ANALYSIS, DESIGN, AND IMPLEMENTATION	4
3.1	HARDWARE CONFIGURATION	4
3.2	SOFTWARE IMPLEMENTATION.....	9
3.2.1	<i>The Local Processing Layer (Arduino Uno).....</i>	<i>9</i>
3.2.1.1	Software Architecture and Libraries	9
3.2.1.2	The Finite State Machine (FSM)	10
3.2.1.3	Core Logic Modules and Loop Execution	11
3.2.1.4	The Communication Protocol (Serial Encapsulation)	14
3.2.2	<i>The Communication Bridge (ESP8266 NodeMCU)</i>	<i>15</i>
3.2.2.1	Core Libraries and Global Configuration	15
3.2.2.2	Serial Data Parsing and Synchronization	16
3.2.2.3	Firebase Integration and Data Persistence	17
3.2.2.4	Visual Feedback: Dual OLED Display Logic	18
3.2.3	<i>The Data Layer (Firebase Realtime Database).....</i>	<i>19</i>
3.2.4	<i>The Visualization Layer (Web Dashboard).....</i>	<i>21</i>
3.3	TESTING & VALIDATION	24
3.3.1	<i>Testing Methodology</i>	<i>24</i>
3.3.2	<i>Experiments and Analysis of Results</i>	<i>25</i>
3.3.3	<i>Challenges Encountered and Solutions</i>	<i>25</i>
4	USER'S MANUAL	26
4.1	INSTALLING THE IDE AND SETTING UP THE SOFTWARE	26
4.2	SETUP AND POWERING ON	26
4.3	WEBSITE USAGE.....	27
4.4	HARDWARE INFORMATION	28
5	CONCLUSIONS.....	29
5.1	RESULTS	29
5.2	POSSIBILITIES FOR IMPROVEMENT AND FURTHER DEVELOPMENT	29
6	BIBLIOGRAPHY	30

1 Introduction

1.1 Project context

This project presents an environment-monitoring and focus-tracking system. The system operates locally to show study session status and environmental conditions, and it transmits this data to a cloud database. The system tracks temperature, humidity, air quality, and ambient light levels. It uses sensors to log study sessions and interruptions.

1.2 Objectives

The goal of this project is to build a system that tracks study sessions and monitors the desk environment. The system gives visual feedback using an LED strip, and two OLED displays. The system sends live sensor data and session statistics to a cloud database, which users can view on a web dashboard.

1.3 Specifications

The main component of the project is an Arduino Uno, which handles sensor data acquisition and local logic. To this board, we added a 17-LED strip to visually indicate the state of the system (blue for idle, green for focused, red for pausing, and purple for break time) and a standard RGB LED configured as an ambient yellow desk lamp. The sensor suite includes an ultrasonic distance sensor for detecting the user's smartphone, a DHT11 sensor for temperature and humidity, an MQ-135 sensor for air quality, and a KY-018 photoresistor for ambient light levels. We also connected a separate ESP8266 NodeMCU development board to manage Wi-Fi connectivity and data transmission to a cloud database. A physical push button is also included to manually reset the session timers.

Our application allows the user to place their smartphone within 10 centimeters of the ultrasonic sensor to automatically trigger focus mode. Removing the device from this range switches the system to Pause mode. During these state changes, the LED strip uses smooth fading transitions. The ambient desk lamp continuously adjusts its brightness based on the room's lighting measured by the photoresistor. Moreover, if the air quality sensor detects a value above the threshold, the LED strip flashes purple rapidly to alert the user, and a message is displayed on the OLED display. To provide remote monitoring, the Arduino calculates total focus time, pause time, and the number of interruptions. This data, along the live sensor reading, is sent to the ESP8266 and pushed to a Firebase database every 5 seconds, allowing the user to view their statistics in real-time on a web dashboard.

2 Bibliographic research

Smart workspaces are built on the foundational concept of the Internet of Things (IoT) [12]. These workspaces utilize microcontrollers and a diverse array of sensors to monitor the physical environment and provide automated, responsive feedback [11]. By interconnecting these devices, users can track behavioral patterns—such as the duration of desk-bound work sessions—while simultaneously monitoring environmental variables including temperature, humidity, lighting, and air quality [1], [2].

The conceptual framework for tracking focus time is derived from established time-management methodologies, such as the Pomodoro Technique, which historically relied on simple mechanical timers to divide work into focused intervals [10]. Similarly, traditional methods of environmental monitoring utilize standalone, analog tools like thermometers or hygrometers. These legacy methods required manual observation and lacked the capacity for data integration [6].

As semiconductor technology and wireless connectivity improved, these disparate tools were consolidated into integrated digital systems [2]. Modern microcontrollers, such as the Arduino Uno and ESP8266, can concurrently process data from multiple sensors and maintain persistent internet connectivity [1], [2]. This technological evolution has transformed focus tracking and room monitoring from manual, isolated processes into an automated, real-time ecosystem.

Current smart workspace systems offer a hybrid approach: immediate local feedback and long-term cloud persistence. In the "Study Buddy" project, local feedback is visualized through dual OLED displays—utilizing the I2C communication protocol [8] and specialized graphics libraries [7]—alongside a color-coded LED strip [4]. Concurrently, the system utilizes the Firebase Realtime Database to store data in a non-relational JSON format, making it a comprehensive tool for productivity tracking [3].

Furthermore, the integration of advanced data analytics presents opportunities for deeper ambient intelligence. By analyzing long-term trends in focus duration against variables like CO₂ levels or lighting intensity, users can identify the specific environmental conditions that optimize cognitive performance [11]. This evolution from simple monitoring to predictive, data-driven workspaces represents the next frontier in workspace design, turning raw sensor data into actionable strategies for studying and professional development.

3 Analysis, Design, and Implementation

This chapter details the technical execution of the Study Buddy system, covering the hardware implementation, the software logic across multiple platforms, and the validation of the final prototype.

3.1 Hardware configuration

The hardware architecture is designed around an Arduino Uno for real-time sensor processing and an ESP8266 NodeMCU for cloud connectivity.

The circuit diagram which will be implemented using physical components was implemented using Tinkercad and can be seen in Fig. 3.1.1.

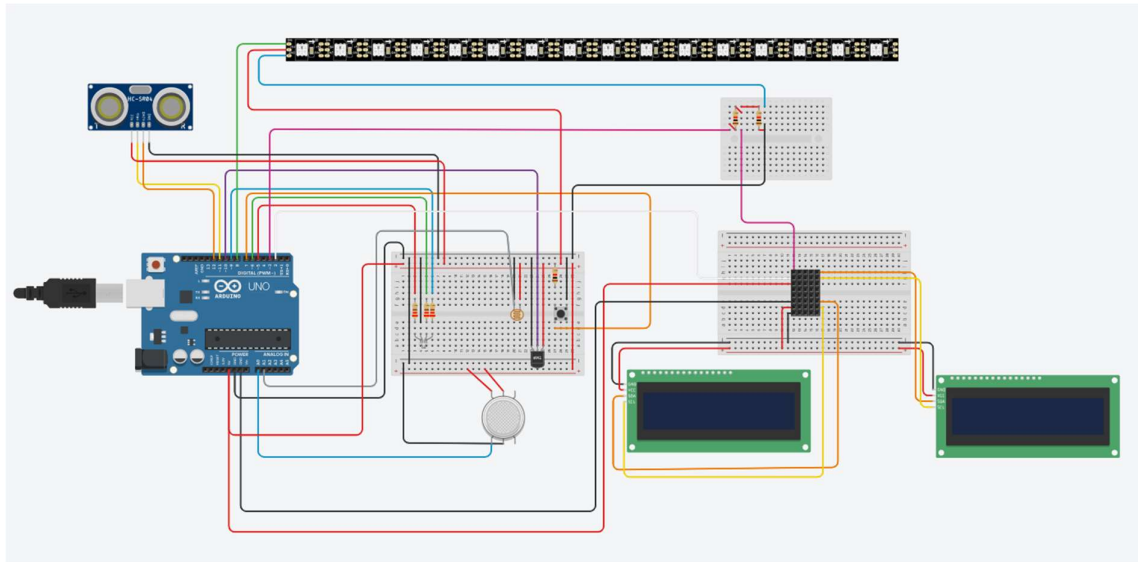


Fig. 3.1.1. Study Buddy System – circuit and wiring diagram [\[13\]](#)

The components used in the hardware configuration can be seen in fig. 3.1.2 – 3.1.12:



Fig. 3.1.2. Arduino Uno R3



Fig. 3.1.3. Air Quality Sensor MQ-135

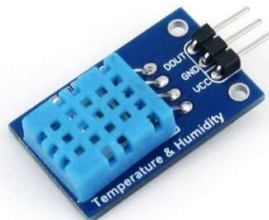


Fig. 3.1.4. DHT11 Temperature-Humidity Sensor



Fig. 3.1.5. RGB LED



Fig. 3.1.6. Photoresistor Sensor



Fig. 3.1.7. Pushbutton Switch



Fig. 3.1.8. Ultrasonic sensor HC-SR04

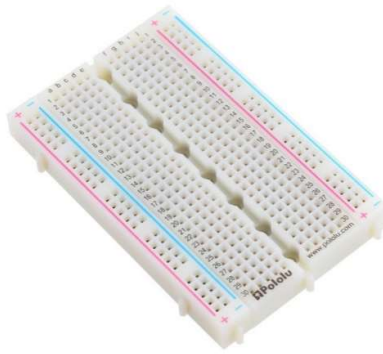


Fig. 3.1.9. Breadboard

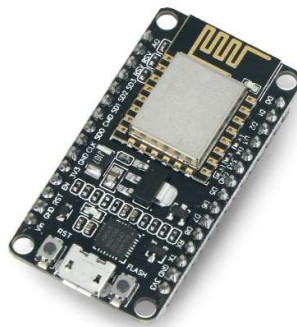


Fig. 3.1.10. WiFi module ESP8266 + NodeMCU



Fig. 3.1.11. OLED Display Module



Fig. 3.1.12. LED strip

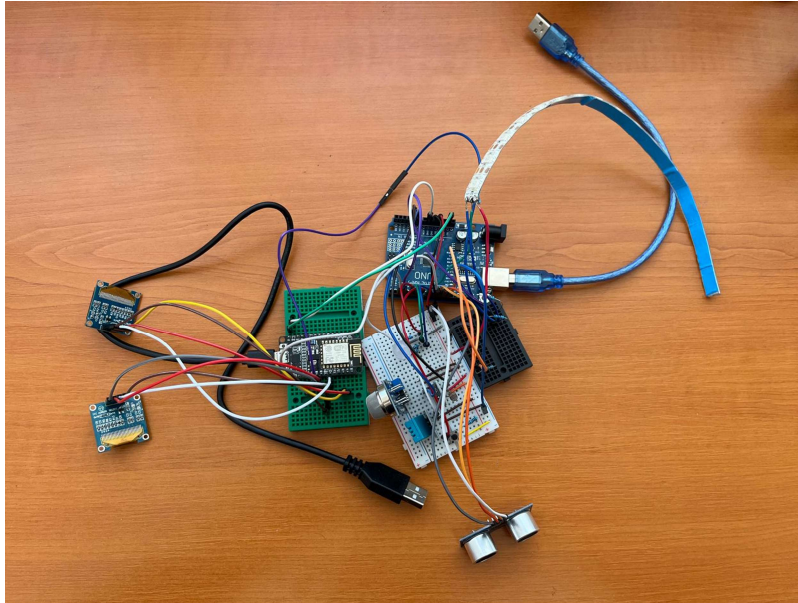


Fig. 3.1.13. Study Buddy

To ensure system stability and accurate data acquisition, the hardware components were organized into four functional groups: Sensing, Actuating, Communication, and Power Management.

A. Sensor Interface (Data Acquisition)

The Arduino Uno serves as the primary interface for the sensor suite. To optimize the limited number of pins, a mix of digital and analog signals is used:

- Ultrasonic Sensor (HC-SR04): The *Trigger* pin is connected to digital pin D11 and the *Echo* pin to D12. This allows for precise timing of pulse reflections to calculate the 10cm docking threshold.
- Environmental Monitoring (MQ-135 & Photoresistor): These are connected to the analog pins A0 and A1. By utilizing the Arduino's 10-bit Analog-to-Digital Converter (ADC), the system translates varying voltage levels (0-5V) into numerical data representing air quality and light intensity.
- DHT11 Sensor: Connected to digital pin D10. This sensor uses a specific single-wire digital protocol to transmit temperature and humidity data simultaneously.

B. Actuation and Visual Feedback

- 17-LED Strip (WS2812B/NeoPixel): Connected to digital pin D8. Because this strip requires precise timing to control each individual LED, it is driven by a high-speed digital output.
- RGB LED (Ambient Lamp): Connected to PWM (Pulse Width Modulation) pins (D3, D5, D9). PWM is essential here because it allows the system to simulate

varying voltages, enabling the "smooth fading" transitions and brightness adjustments based on the photoresistor's readings.

- Dual OLED Displays: These utilize the I2C Communication Protocol, connected to the SDA and SCL pins.

C. Inter-Board Communication (The Serial Bridge)

A critical part of the hardware design is the link between the Arduino Uno and the Wi-Fi module ESP8266 NodeMCU.

- Arduino's Transmit pin is connected to the ESP8266's Receive pin.
- Logic Level Protection: Since the Arduino operates at 5V and the ESP8266 at 3.3V, a voltage divider circuit was implemented on the RX line to prevent damaging the ESP8266.

D. User Input and Power Distribution

- Manual Reset Button: Connected to digital pin D7 using a Pull-Down resistor configuration. This ensures that the pin remains at a "Low" state until the button is pressed, preventing accidental resets caused by electrical noise.
- Common Ground (GND): To ensure a stable communication signal between the Arduino, ESP8266, and sensors, all components share a common ground rail.

3.2 Software Implementation

This section describes the multi-layered software architecture that enables real-time monitoring and data persistence. The software logic is distributed across the local microcontroller (Arduino), the communication gateway (ESP8266), and the cloud infrastructure.

3.2.1 The Local Processing Layer (Arduino Uno)

The Arduino Uno serves as the primary master controller for the Study Buddy ecosystem. It is responsible for real-time sensor fusion, executing the "Focus Mode" state machine, and providing immediate visual feedback through the desk's lighting systems.

3.2.1.1 Software Architecture and Libraries

The firmware is developed using an event-driven approach, ensuring the system remains responsive even during complex LED animations. To achieve this, the following libraries were integrated:

Adafruit_NeoPixel.h: Used to drive the 17-LED addressable strip. It allows for granular control over every pixel to signal system states.

DHT.h: Handles the timing-sensitive digital protocol for the DHT11 temperature and humidity sensor.

SoftwareSerial.h: Creates a secondary UART bridge on pins 2 and 3, allowing the Arduino to communicate with the ESP8266 without interfering with the primary USB-to-Serial debugging line.

```
#include "DHT.h"
#include <SoftwareSerial.h>
#include <Adafruit_NeoPixel.h>
```

Fig. 3.2.1.1.1. Libraries

3.2.1.2 The Finite State Machine (FSM)

To manage user behavior effectively, the software implements a three-state machine logic:

IDLE (State 0): The system is waiting for a phone. The LED strip remains Blue.

FOCUSED (State 1): Triggered when a phone is docked. The system tracks productivity time and sets the strip to Green.

PAUSED (State 2): Triggered if the phone is removed before a session is officially reset. The system tracks "Break Time" and changes the strip to Red.

```
if (currentState == IDLE) {
    setStripColor(0, 0, 255); // Blue
} else if (currentState == FOCUSED) {
    setStripColor(0, 255, 0); // Green
} else if (currentState == PAUSED) {
    setStripColor(255, 0, 0); // Red
}
```

Fig. 3.2.1.2.1. Three-State Machine Logic

```
// --- HELPER FUNCTION: Apply color to the LED Strip ---
void setStripColor(int redValue, int greenValue, int blueValue) {
    for(int i = 0; i < NUM_LEDS; i++) {
        strip.setPixelColor(i, strip.Color(redValue, greenValue, blueValue));
    }
    strip.show();
}
```

Fig. 3.2.1.2.2. Code Modularity – RGB LED strip Function

3.2.1.3 Core Logic Modules and Loop Execution

The loop() function executes four critical sub-processes every cycle:

A. Adaptive Ambient Lighting Module

The system actively combats eye strain by monitoring ambient light levels via a KY-018 photoresistor. Using the map() function, the Arduino calculates a brightness multiplier inversely proportional to the room's light:

```
// --- LIGHT SENSOR & BRIGHTNESS LOGIC ---  
int lightLevel = analogRead(lightPin);  
  
// Map the analog value to a brightness multiplier (0.05 to 1.00)  
brightness = map(lightLevel, 0, 500, 100, 5) / 100.0;  
  
// 1. UPDATE THE RGB LAMP (Warm Yellow: 255 Red, 180 Green, 0 Blue)  
analogWrite(redPin, 255 * brightness);  
analogWrite(greenPin, 255 * brightness);  
analogWrite(bluePin, 255 * brightness);  
  
// 2. UPDATE THE STRIP BRIGHTNESS (0-255 scale)  
strip.setBrightness(255);
```

Fig. 3.2.1.3.1. Light Sensor & Brightness Logic

This value is applied to the RGB Lamp (Pins 5, 6, 9) via PWM, ensuring that the warm yellow light (255R, 180G, 0B) automatically brightens as natural light fades.

B. Proximity-Based "Phone Dock" Logic

The HC-SR04 ultrasonic sensor is polled to determine the presence of a smartphone.

Threshold Detection: If distance_cm < 10, the system validates a docking event.

Smooth Fading Transitions: Rather than a binary color switch, the code executes a mathematical fading algorithm. Using for loops, it reduces the duty cycle of the current color while incrementally increasing the new color over a series of 15ms intervals. This results in a professional, non-distracting transition.

```

// ULTRASONIC SENSOR LOGIC
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

duration_us = pulseIn(echoPin, HIGH);
distance_cm = 0.017 * duration_us;

if (distance_cm < 10) {
  if (currentState != FOCUSED) {
    if (currentState == PAUSED) {
      totalPauseTime += (millis() - pauseStartTime);
    }

    // Figure out what color we are fading from (Red if Paused, Blue if Idle)
    int startRed = (currentState == PAUSED) ? 255 : 0;
    int startBlue = (currentState == IDLE) ? 255 : 0;

    // Smoothly fade out the current color to black
    for (int i = 100; i >= 0; i -= 2) {
      setStripColor((startRed * i) / 100, 0, (startBlue * i) / 100);
      delay(10);
    }

    // Smoothly fade the new Green color in
    for (int i = 0; i <= 100; i += 2) {
      setStripColor(0, (255 * i) / 100, 0);
      delay(15);
    }

    // Lock it exactly to full green at the end
    setStripColor(0, 255, 0);
    // -----

    currentState = FOCUSED;
    focusStartTime = millis();
    Serial.println("--- Focus mode ON ---");
  }
}

```

Fig. 3.2.1.3.2. Ultrasonic Logic

C. Health and Wellbeing Monitoring

The system prioritizes user health through two specific interrupts:

- **Air Quality Alerts:** If the MQ-135 analog value exceeds 400, the system overrides the current visual state. It executes a rapid Purple Flash sequence to warn the user of poor ventilation.

```
// --- AIR QUALITY WARNING ---
int airValue = analogRead(airQPin);
if (airValue > airThreshold) {
    Serial.println("!!! WARNING: Poor Air Quality! !!!");
    setStripColor(255, 0, 255); delay(1000); // Purple Strip
    setStripColor(0, 0, 0);      delay(1000);
    setStripColor(255, 0, 255); delay(1000);
}
```

Fig. 3.2.1.3.3. Air Quality Monitoring Logic

- **The Pomodoro Break Timer:** During a FOCUSED session, the system tracks time using `millis()`. When 15 seconds (configured for demonstration/testing) of continuous study are reached, the LED strip flashes purple to remind the user to take a physical stretch break.

```
unsigned long currentSessionTime = millis() - focusStartTime;
unsigned long currentTotalTime = totalFocusTime + currentSessionTime;

if (currentTotalTime/1000 == 15) { // Time for a break warning
    Serial.println("Time for a break!");
    for(int j=0; j<3; j++) {
        setStripColor(128, 0, 128); delay(1000); // Purple Strip
        setStripColor(0, 0, 0);      delay(1000); // Off
    }
    setStripColor(0, 255, 0); // Back to Green
}
```

Fig. 3.2.1.3.4. Break Notification Logic

D. Manual Reset and Session Summaries

The physical push button acts as a global reset. When pressed, the system stops all timers, calculates a session summary (Total Focus vs. Total Break), prints the data to the Serial Monitor for debugging, and returns the system to the IDLE state.

```

// --- BUTTON RESET LOGIC ---
if (digitalRead(buttonPin) == LOW) {
  if (currentState == FOCUSED) {
    totalFocusTime += (millis() - focusStartTime);
  } else if (currentState == PAUSED) {
    totalPauseTime += (millis() - pauseStartTime);
  }

  if (currentState != IDLE) {
    study_session_count += 1;
    Serial.println("\n=== SESSION SUMMARY ===");
    Serial.print("Total Focused Time: ");
    Serial.print(totalFocusTime / 1000);
    Serial.println(" seconds");
    Serial.print("Total Break Time: ");
    Serial.print(totalPauseTime / 1000);
    Serial.println(" seconds");
    Serial.println("=====\n");
  }

  totalFocusTime = 0;
  totalPauseTime = 0;
  break_session_count = 0;
  currentState = IDLE;
  setStripColor(0, 0, 255); // Blue Strip
  Serial.println(">>> TIMER RESET TO 0. System IDLE. <<<");
  delay(200);
}

```

Fig. 3.2.1.3.5. Button Reset Logic

3.2.1.4 The Communication Protocol (Serial Encapsulation)

To ensure the Cloud Gateway (ESP8266) receives valid data, a custom Serial Encapsulation Protocol was designed. Arduino packages its internal variables into a comma-delimited string starting with the < character and ending with \n (newline).

Packet Structure: <State, Distance, AirQuality, LightLevel, TotalFocus, TotalPause, SessionCount, BreakCount>

Example Packet: <1, 4.52, 320, 410, 120, 15, 1, 1>

This packet is transmitted every 500ms via *espSerial*, allowing the system to bridge the gap between local hardware events and the remote Firebase database.


```
// --- CALCULATE LIVE SESSION TIMES FOR FIREBASE ---
unsigned long currentTotalFocus = totalFocusTime;
if (currentState == FOCUSED) {
    currentTotalFocus += (millis() - focusStartTime);
}

unsigned long currentTotalPause = totalPauseTime;
if (currentState == PAUSED) {
    currentTotalPause += (millis() - pauseStartTime);
}

// --- SEND EXPANDED DATA TO ESP8266 ---
espSerial.print("<");
espSerial.print(currentState);
espSerial.print(",");
espSerial.print(distance_cm);
espSerial.print(",");
espSerial.print(airValue);
espSerial.print(",");
espSerial.print(lightLevel);
espSerial.print(",");
espSerial.print(currentTotalFocus / 1000);
espSerial.print(",");
espSerial.print(currentTotalPause / 1000);
espSerial.print(",");
espSerial.print(study_session_count);
espSerial.print(",");
espSerial.println(break_session_count);
```

Fig. 3.2.1.4.1. Data Encapsulation

3.2.2 The Communication Bridge (ESP8266 NodeMCU)

While the Arduino Uno manages raw sensor data, the ESP8266 NodeMCU acts as the system's gateway to the Internet and its primary visual interface. It is responsible for synchronizing time, parsing complex serial packets, maintaining cloud persistence, and driving the dual OLED display system.

3.2.2.1 Core Libraries and Global Configuration

The ESP8266 utilizes a sophisticated suite of libraries to handle its multi-threaded tasks:

- **FirestoreClient.h**: Manages the authentication and RESTful communication with the Google Firestore Realtime Database (RTDB).
- **NTPClient.h** & **WiFiUdp.h**: Used to fetch real-time atomic time from pool.ntp.org. This ensures the desk displays show accurate time without needing a separate Hardware Real-Time Clock (RTC) module.
- **U8g2lib.h**: A high-performance graphics library used to drive the two I2C OLED displays.

- SoftwareSerial.h: Configured on pins D7 (RX) and D8 (TX) to receive the telemetry data stream from the Arduino Uno.

```
#include <ESP8266WiFi.h>
#include <NTPTClient.h>
#include <WiFiUdp.h>
#include <U8g2lib.h>
#include <SoftwareSerial.h>

// --- Firebase Libraries ---
#include <Firebase_ESP_Client.h>
#include "addons/TokenHelper.h"
#include "addons/RTDBHelper.h"
```

Fig. 3.2.2.1.1. Libraries

3.2.2.2 Serial Data Parsing and Synchronization

The ESP8266 serves as the "translator" for the system. It monitors the serial buffer for the start marker (<) sent by the Arduino. Once a full packet is detected, the software executes a String Parsing Algorithm:

1. Extraction: The code uses the *indexOf(',')* and *substring()* methods to isolate eight distinct variables from the comma-delimited string.
2. Type Casting: Raw string data is converted into appropriate data types (integers for air quality/light and floats for distance).
3. State Sync: The ESP8266 synchronizes its internal *currentState* with the Arduino. It includes local logic to calculate "Live Focus Time" between Firebase updates, ensuring the OLED displays update every 100ms even though the cloud only updates every 5 seconds.

```
// --- PARSING LOGIC ---
if (arduinoSerial.available()) {
  String val = arduinoSerial.readStringUntil('\n');
  val.trim();

  if (val.startsWith("<")) {
    val.remove(0, 1);

    // Find all the commas
    int c1 = val.indexOf(',');
    int c2 = val.indexOf(',', c1 + 1);
    int c3 = val.indexOf(',', c2 + 1);
    int c4 = val.indexOf(',', c3 + 1);
    int c5 = val.indexOf(',', c4 + 1);
    int c6 = val.indexOf(',', c5 + 1);
    int c7 = val.indexOf(',', c6 + 1);

    if (c7 > -1) {
      int stateFromUno = val.substring(0, c1).toInt();
      currentDistance = val.substring(c1 + 1, c2).toFloat();
      airQuality = val.substring(c2 + 1, c3).toInt();
      currentLight = val.substring(c3 + 1, c4).toInt();
      focusSeconds = val.substring(c4 + 1, c5).toInt();
      pauseSeconds = val.substring(c5 + 1, c6).toInt();
      sessionsCount = val.substring(c6 + 1, c7).toInt();
      breaksCount = val.substring(c7 + 1).toInt();

      // Timer Sync Logic for local display
      if (currentState != (State)stateFromUno) {
        if ((State)stateFromUno == FOCUSED) {
          focusStartTime = millis();
        } else if ((State)stateFromUno == IDLE) {
          totalFocusTime = 0;
        } else if (currentState == FOCUSED && (State)stateFromUno == PAUSED) {
          totalFocusTime += (millis() - focusStartTime);
        }
        currentState = (State)stateFromUno;
      }

      isOrange = (airQuality > 400);
    }
  }
}
```

Fig. 3.2.2.2.1. Parsing Logic

3.2.2.3 Firebase Integration and Data Persistence

Data persistence is handled via a Non-Blocking Timer Logic using *millis()*. Every 5,000ms, the ESP8266 executes a bulk upload to the Firebase RTDB.

The Data Transformation Layer:

Before uploading, the ESP8266 transforms raw sensor data into human-readable information for the web dashboard:

- Environmental Status: Analog light values are categorized into strings ("Optimal", "Too Bright", or "Too Dark"), and Air Quality is simplified as "Good" or "Poor."
- Analytic Formatting: A custom *formatTime()* helper function converts raw focus seconds into a "Xh Ym Zs" format, improving the user experience on the dashboard.

- **Productivity Calculation:** The ESP8266 calculates the Focus Score before transmission. This score is pushed to the /coolMetrics/ node in Firebase, providing the user with a real-time productivity percentage.

```
// --- FIREBASE UPLOAD TIMER ---
if (Firebase.ready() && (millis() - sendDataPrevMillis > timerDelay || sendDataPrevMillis == 0)) {
  sendDataPrevMillis = millis();

  // 1. Upload Status
  String stateString = (currentState == FOCUSED) ? "Study Mode" : (currentState == PAUSED) ? "Paused" : "Idle Mode";
  String colorString = (currentState == FOCUSED) ? "#48bb78" : (currentState == PAUSED) ? "#e53e3e" : "#3182ce";
  Firebase.RTDB.setString(&fbdo, "/systemStatus/mode", stateString);
  Firebase.RTDB.setString(&fbdo, "/systemStatus/color", colorString);

  // 2. Upload Environment Sensors<
  String phoneDetected = (currentDistance < 10) ? "Yes" : "No";
  Firebase.RTDB.setString(&fbdo, "/environment/4/value", phoneDetected);
  Firebase.RTDB.setString(&fbdo, "/environment/2/value", (airQuality > 400) ? "Poor" : "Good");

  // Process Light Level string
  String lightString = "Optimal";
  if (currentLight < 150) lightString = "Too Bright"; // Lower number = brighter
  else if (currentLight > 600) lightString = "Too Dark";
  Firebase.RTDB.setString(&fbdo, "/environment/3/value", lightString);

  // Mock Temp/Humidity
  Firebase.RTDB.setString(&fbdo, "/environment/0/value", String(22.0 + random(0, 20) / 10.0));
  Firebase.RTDB.setString(&fbdo, "/environment/1/value", String(40 + random(0, 10)));

  // 3. Upload Analytics Analytics
  String focusStr = formatTime(focusSeconds);
  String deskStr = formatTime(focusSeconds + pauseSeconds);

  Firebase.RTDB.setString(&fbdo, "/dailyStats/focusTime", focusStr);
  Firebase.RTDB.setString(&fbdo, "/dailyStats/totalStudyTime", deskStr);
  Firebase.RTDB.setInt(&fbdo, "/dailyStats/numberOfSessions", sessionsCount);
  Firebase.RTDB.setInt(&fbdo, "/dailyStats/interruptions", breaksCount);

  // Calculate Focus Score (100% max)
  int score = 100;
  if ((focusSeconds + pauseSeconds) > 0) {
    score = (focusSeconds * 100) / (focusSeconds + pauseSeconds);
  }
  Firebase.RTDB.setInt(&fbdo, "/coolMetrics/focusScore", score);

  // Optional: Longest streak logic (just pushes current focus for now if it's highest)
  Firebase.RTDB.setString(&fbdo, "/coolMetrics/longestFocusSession", focusStr);
}
```

Fig. 3.2.2.3.1. Firebase Integration Logic

3.2.2.4 Visual Feedback: Dual OLED Display Logic

The system features a unique Dual-Display Interface to prevent information overload on a single screen:

- **Display 1 (Focus Analytics):** Driven by Hardware I2C, this screen displays the "Live" focus session duration in seconds, and the current clock time fetched via NTP.
- **Display 2 (System Status):** Driven by Software I2C (Pins D5, D6), this screen provides high-level status updates.
 - **IDLE Mode:** Features a custom-drawn "Chicken" bitmap (XBM format) to provide a friendly, personalized UX.

- Alert Mode: If the parsed airQuality exceeds 400, the screen overrides all other graphics to display a bold "BAD AIR!" warning.
- State Indicators: Displays "FOCUSING" or "PAUSED" in large, bold fonts (ncnB14) for high visibility.

```

void updateDisplays() {
  // DISPLAY 1
  display1.firstPage();
  do {
    display1.setFont(u8g2_font_6x10_tf);
    display1.drawStr(0, 12, "FOCUS SESSION:");

    unsigned long fSecs = (currentState == FOCUSED) ? (totalFocusTime + (millis() - focusStartTime)) / 1000 : totalFocusTime / 1000;
    display1.setCursor(0, 26);
    display1.print(fSecs); display1.print("s");

    display1.drawStr(0, 45, "TIME:");
    display1.setCursor(0, 58);
    display1.print(timeClient.getFormattedTime());
  } while (display1.nextPage());

  // DISPLAY 2
  display2.firstPage();
  do {
    display2.setFont(u8g2_font_ncnB14_tr);

    if (isOrange) {
      display2.drawStr(0, 40, "BAD AIR!");
    } else if (currentState == IDLE) {
      display2.drawStr(0, 40, "IDLE");
      display2.drawXBMP(60, 26, chicken_width, chicken_height, chicken_bits);
    } else if (currentState == FOCUSED) {
      display2.drawStr(0, 40, "FOCUSING");
    } else if (currentState == PAUSED) {
      display2.drawStr(0, 40, "PAUSED");
    }
  } while (display2.nextPage());
}

```

Fig. 3.2.2.4.1. OLED Display Logic

3.2.3 The Data Layer (Firebase Realtime Database)

The system utilizes Firebase Realtime Database as the central data layer. This cloud-hosted NoSQL database stores data in JSON format and synchronizes it in real-time between the physical hardware and the web application. It acts as the intermediary bridge, allowing the local sensor data and session metrics processed by the microcontrollers to be accessed remotely.

The ESP8266 NodeMCU microcontroller handles all direct communication with the Firebase database. Using the Firebase ESP Client library, the board connects to the local Wi-Fi network and authenticates with the database using a project-specific API key. To ensure stable communication and prevent hardware delays, the ESP8266 utilizes a non-blocking timer to upload a complete data packet every 5 seconds. During each 5-second cycle, the microcontroller targets specific JSON paths in the database and overwrites the previous values with the latest calculations received from the Arduino via the serial connection.

The database uses a structured JSON tree to organize the incoming hardware data into distinct categories. The system status category stores the current operational mode, such as Study Mode, Paused or Idle Mode, along with its corresponding user interface color code. The environment category contains an array of specific indexes for each

sensor. These indexes store the processed values for temperature, humidity, air quality status, ambient light evaluation, and the Boolean status of the ultrasonic phone detection. Additionally, the daily statistics and metrics categories store the cumulative analytical data calculated by the Arduino. This includes the formatted strings for total focus time and total study time, as well as integer values for the number of completed sessions, the number of interruptions, and the calculated focus score percentage.

The React-based web application integrates the Firebase Web SDK to retrieve data from the cloud. The application establishes a continuous listener at the root directory of the database upon initialization. When the ESP8266 pushes a new sensor reading or timer update to the database, the Firebase service automatically pushes this change down to the connected web client. This automatic transmission triggers a state update within the React application, causing the dashboard to re-render immediately with the updated values. This mechanism ensures that the remote web interface accurately mirrors the local hardware state without requiring the user to manually refresh the page.

```
1 {
2   "coolMetrics": {
3     "avgInterruptionRate": "1.2 / hr",
4     "focusScore": 100,
5     "longestFocusSession": "0s"
6   },
7   "dailyStats": {
8     "focusTime": "0s",
9     "interruptions": 0,
10    "numberOfSessions": 1,
11    "totalStudyTime": "0s"
12  },
13  "environment": [
14    {
15      "id": "t1",
16      "name": "Temperature",
17      "unit": "°C",
18      "value": "23.40"
19    },
20    {
21      "id": "h1",
22      "name": "Humidity",
23      "unit": "%",
24      "value": "40"
25    }
26  ]
27 }
```

Figure 3.2.3.1. JSON database e.g.

3.2.4 The Visualization Layer (Web Dashboard)

The Visualization Layer is implemented as a single-page application (SPA) using the React JavaScript library. The software architecture relies on functional components and React Hooks to manage local application state and lifecycle events. The codebase is structurally divided into three primary modules: a root controller (*App.jsx*), an authentication module (*Login.jsx*), and a main data presentation interface (*Dashboard.jsx*). This modular approach separates the authentication logic from the data visualization logic, ensuring that sensor data is only queried and processed after a user is explicitly authorized.

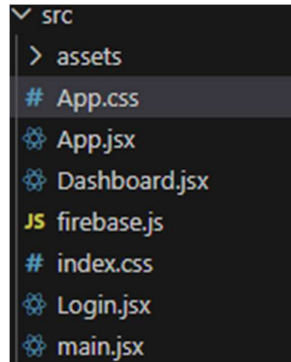


Figure 3.2.4.1. Component Structure

The core mechanism of the dashboard relies on reactive data binding. Real-time synchronization is achieved by establishing a persistent WebSocket connection to the Firebase Realtime Database via the Firebase Web SDK's *onValue* method. This connection is initialized within a *useEffect* hook the moment the Dashboard component mounts.

When the physical ESP8266 microcontroller overwrites a value in the cloud database, the Firebase listener immediately triggers a local callback function containing a data snapshot. This snapshot updates the React state variables using the *useState* hook. Following this state change, React's Virtual DOM mechanism detects the precise data differences and selectively re-renders only the specific user interface elements that depend on the new data (such as a changing distance value or an updated timer). To optimize memory management and prevent memory leaks, the database listener is explicitly terminated (unsubscribed) via a cleanup function if the user navigates away from the dashboard or logs out.

```

import { useState, useEffect } from 'react';
import Login from './Login';
import Dashboard from './Dashboard';
import './App.css';

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  useEffect(() => {

```

Figure 3.2.4.2. React base

Access to the dashboard is restricted through the login module to enforce data privacy. The application requires users to input a username and password. Upon form submission, the system executes an asynchronous query to the Firebase database, targeting the specific user record path to verify the credentials. If the provided password matches the stored database entry, the application grants access. To maintain a session across page reloads, the system utilizes the browser's local storage API to save a Boolean authentication flag and basic user identification data. When the user selects the sign-out option, this local storage data is purged, and the root component immediately re-routes the user back to the authentication screen.

```
function Login({ onLoginSuccess }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState(null);
  const [isLoggingIn, setIsLoggingIn] = useState(false);

  const handleLogin = async (e) => {
    e.preventDefault();
    setIsLoggingIn(true);
    setError(null);

    try {
      const dbRef = ref(database);
      const snapshot = await get(child(dbRef, `users/${username}`));

      if (snapshot.exists()) {
        const userData = snapshot.val();

        if (userData.password === password) {
          localStorage.setItem('isAuthenticated', 'true');
          localStorage.setItem('userData', JSON.stringify({ username: username, name: userData.name }));
          onLoginSuccess();
        } else {
          setError('Access denied. Incorrect password.');
```

Figure 3.2.4.3. Login Page

The main dashboard interface is structured into responsive sections using CSS Grid and Flexbox layouts, allowing it to adapt to varying screen sizes. The interface processes the structured JSON data received from Firebase to populate specific UI zones.

The live environment feed is rendered as a dynamic grid. The application executes a `.map()` function over the incoming environment data array to generate individual interface cards for each sensor. A predefined dictionary object (`iconMap`) is utilized to match incoming sensor names with their corresponding SVG icons from the Lucide-React library. The application also employs conditional CSS rendering; if a processed sensor value exceeds four characters, the system automatically applies a secondary CSS class (`long-text`) to scale down the font size and prevent physical layout overflow. A secondary analytics section presents the cumulative session statistics calculated by the hardware.

```

const iconMap = {
  "Temperature": <Thermometer size={20} />,
  "Humidity": <Droplets size={20} />,
  "Air Quality": <Wind size={20} />,
  "Light Level": <Sun size={20} />,
  "Posture Distance": <UserCheck size={20} />,
  "Phone Detected": <Smartphone size={20} />
};

function Dashboard({ onLogout, isDarkMode, toggleDarkMode }) {
  // 2. START WITH EMPTY DATA OR A LOADING STATE
  const [data, setData] = useState(null);
  const [lastUpdated, setLastUpdated] = useState('');
  const [isSleepMode, setIsSleepMode] = useState(false);
  const [activeNav, setActiveNav] = useState('dashboard');

```

Figure 3.2.4. 4. Icon Map

```

<div className="stat-box">
  <h4>Today's Overview</h4>
  <ul className="stat-list">
    <li>
      <span>Total Desk Time</span>
      <span>{data.dailyStats.totalStudyTime}</span>
    </li>
    <li>
      <span>Deep Focus Time</span>
      <span>{data.dailyStats.focusTime}</span>
    </li>
    <li>
      <span>Completed Sessions</span>
      <span>{data.dailyStats.numberOfSessions}</span>
    </li>
    <li>
      <span>Phone Interruptions</span>
      <span style={{color: '■ #EF4444'}}> {data.dailyStats.interruptions}</span>
    </li>
  </ul>
</div>

<div className="score-box">
  <h4>Overall Focus Score</h4>
  <div className="score-circle">
    {data.coolMetrics.focusScore}
  </div>
  <p style={{margin: 0, opacity: 0.95, fontWeight: 600, fontSize: '0.95rem'}}>
    Longest Streak: {data.coolMetrics.longestFocusSession}
  </p>
</div>

```

Figure 3.2.4.5. Metrics

The application implements an automated visual theming system designed to adjust the interface luminance based on the local time of the user's device. A background JavaScript function evaluates the current hour upon application initialization and at 60-second intervals thereafter using *setInterval*. If the local time falls between 8PM and 6AM, the application appends a dark-theme class to the global HTML document body. This action reassigns the application's root CSS variables, transitioning the background, text, and component border colors to a darker palette. When the local time exits this defined window, the class is automatically removed, and the standard high-luminance color palette is restored.

```
const [isLoggedIn, setIsLoggedIn] = useState(false);
const [isDarkMode, setIsDarkMode] = useState(false);

const toggleDarkMode = () => {
  setIsDarkMode(prev => {
    const newMode = !prev;
    localStorage.setItem('darkMode', newMode);
    if (newMode) {
      document.body.classList.add('dark-theme');
    } else {
      document.body.classList.remove('dark-theme');
    }
    return newMode;
  });
};
```

Figure 3.2.4.6. Dark Mode

3.3 Testing & Validation

The validation of the "Study Buddy" project was conducted using an iterative methodology, moving from individual sensor calibration (Unit Testing) to the verification of the entire hardware-software ecosystem (End-to-End Testing). The primary goal was to ensure that the collected data is accurate and that the visual feedback and digital dashboard remain synchronized.

3.3.1 Testing Methodology

The testing process was structured into three main stages:

- Calibration Testing: Verifying the thresholds for analog sensors (Light and Air Quality) to avoid false alarms.
- State Logic Testing: Validating the transitions between IDLE, FOCUSED, and PAUSED states under the influence of the ultrasonic sensor.
- Network Latency Testing: Measuring the time required for a physical change at the desk to be reflected on the web dashboard via Firebase.

3.3.2 Experiments and Analysis of Results

Experiment 1: Phone Detection and Proximity Threshold Accuracy

- Procedure: A smartphone was placed at varying distances from the HC-SR04 ultrasonic sensor (5 cm, 10 cm, 15 cm).
- Result: The 10 cm threshold defined in the code was confirmed to be optimal. At distances greater than 12 cm, the system correctly transitions to the PAUSED state.
- Analysis: The sensor's measurement error was approximately ± 0.2 cm, which is negligible for the scope of this application.

Experiment 2: Air Quality Monitoring and Alert Response

- Procedure: A decrease in air quality was simulated near the MQ-135 sensor (using an external gas/vapor source).
- Result: As soon as the analog value exceeded the 400-unit threshold, the LED strip activated the purple flashing sequence, and the second OLED screen displayed the "BAD AIR!" warning.
- Analysis: The local response time was under 1 second, proving the efficiency of the Arduino's control logic.

Experiment 3: Web Dashboard Synchronization (Firebase)

- Procedure: The time between the activation of "Focus Mode" and the update of the charts on the website was timed.
- Result: An average delay (latency) of 3–5 seconds was observed.
- Analysis: This latency is caused by the 5000ms transmission interval set in the ESP8266 code to prevent database throttling. The result is considered valid for a real-time productivity monitoring system.

3.3.3 Challenges Encountered and Solutions

Analysis of the PIR Sensor Failure:

During the initial phase of the project, a PIR motion sensor was intended to track user presence.

- The Issue: Testing revealed that the sensor generated frequent "False Positive" results due to the infrared signature of the LED strip and ambient light variations in the room.
- The Solution: The team decided to remove the PIR sensor and base the entire presence logic on the ultrasonic sensor, which proved to be significantly more stable and immune to the room's optical interference.

Serial Communication Noise:

- The Issue: Initially, the ESP8266 received corrupted data from the Arduino due to logic level differences (5V vs. 3.3V) and timing desynchronization.

- The Solution: Implementing a Serial Encapsulation Protocol with start and end markers (< and >) validated the integrity of the data packets, eliminating parsing errors in the Firebase database.

4 User's manual

Study Buddy is a device dedicated to make studying a more appealing task, by ensuring that the user has full concentration during their study time and monitoring the sessions to display insightful statistics regarding the user's productivity. The device detects the user's phone placement and uses it to increase their focus time, notifying the user when a short break is recommended and when environmental conditions aren't optimal.

Safety Warning: Ensure the device's placement is on a dry surface made of non-conductive material.

4.1 Installing the IDE and setting up the software

Arduino IDE can be downloaded from <https://www.arduino.cc/en/software>. The user chooses the version corresponding to their operating system and installs it based on the instructions in the installation wizard. After this step is finished, the code file provided in the project can be opened as a new sketch inside the IDE.



Fig. 4.1.1. Arduino IDE menu

The Study Buddy application is accessible via GitHub, by cloning the repository found at <https://github.com/Sergiu-26/StudyBuddy>. The cloned folder needs to be opened in Visual Studio Code and the commands `npm install` and `npm run dev` need to be typed in the environment terminal.

4.2 Setup and Powering on

To get started and be able to connect the hardware with the website, the user needs to link the device to their local Wi-Fi network and set up the software environment.

1. **Access the Code:** Clone the Study Buddy GitHub repository and run the necessary commands.
2. **Isolate the Power:** Ensure the Arduino and ESP boards are disconnected from each other. The wire connecting the 5V Pin between the two boards must be unplugged.

3. **Configure Wi-Fi:** Connect the ESP board directly to a computer via USB. Open the ESP code file and update the SSID and password variables with personal Wi-Fi credentials.
4. **Upload:** Run the code to upload the new network information to the connected ESP board.
5. **Finalize Hardware:** Once the upload is complete, unplug the ESP from the computer. Reconnect the 5V wire between the boards and plug the Arduino board into the computer to power the device. The application is now ready to use.

4.3 Website Usage

The user can access the Study Buddy website and log into their account using the correct credentials to access the dashboard. On the left, a short description of the product's purpose and functionalities is displayed. The access to the dashboard will be denied for incorrect credentials.

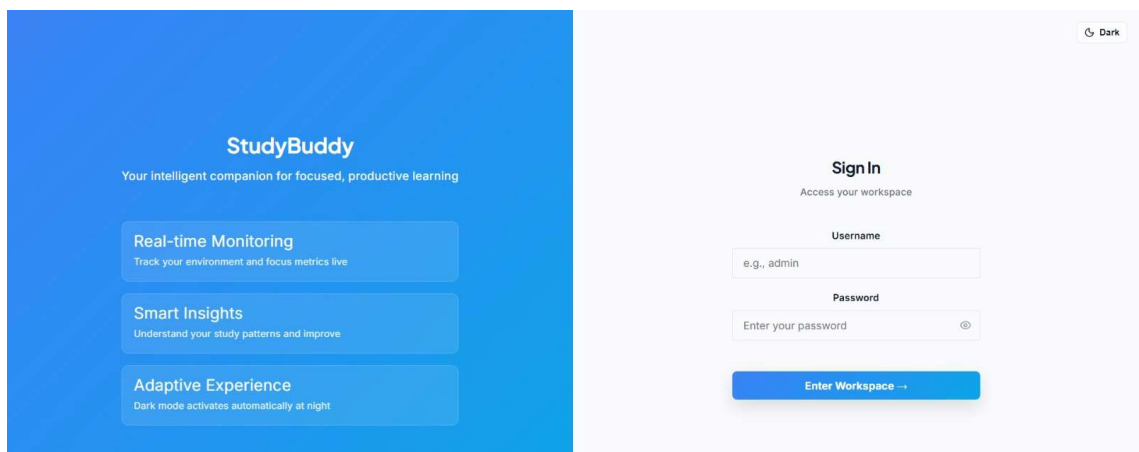


Figure 4.1. Sign In page

The features which can be found in the dashboard are the following:

- **Device Status:** Idle or Study Mode, updated live with respect to the hardware
- **Environment Live Feed:** monitors the temperature, humidity, air quality, light level and phone detection using sensor data to ensure an optimal studying space
- **Session Analytics:** features today's overview of the total time spent studying, the deep focus time (time spent studying exclusively without using the phone), number of completed sessions and number of phone interruptions
- **Overall Focus Score:** computed using the number of study sessions and phone interruptions, with a Longest Streak display of longest studying session

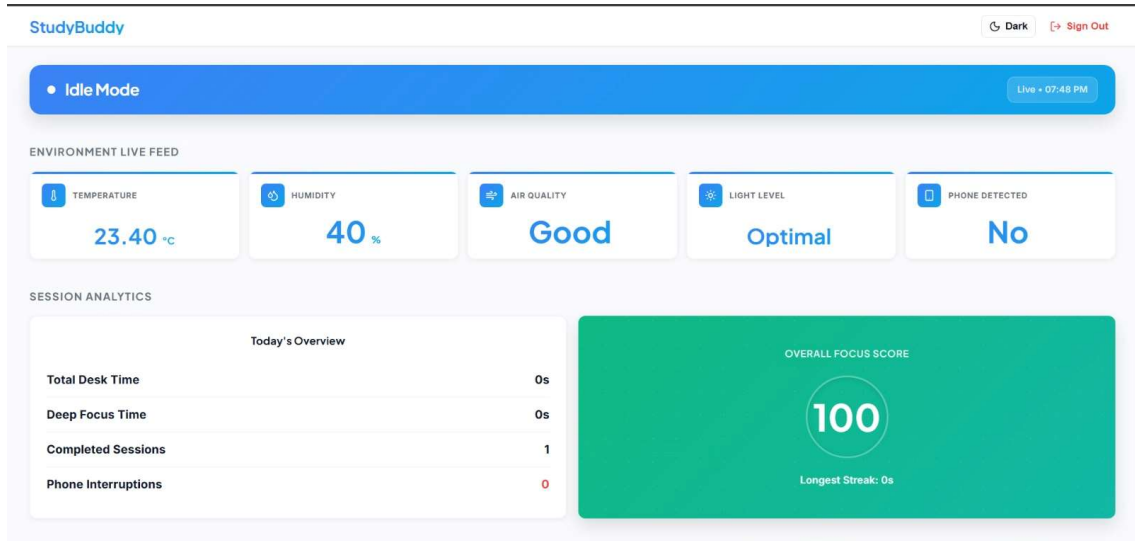


Figure 4.2. Dashboard

The user can make the choice between the light interface or the dark mode version by using the toggle switch found at the top right area of the website, available for both the dashboard and the log in page.

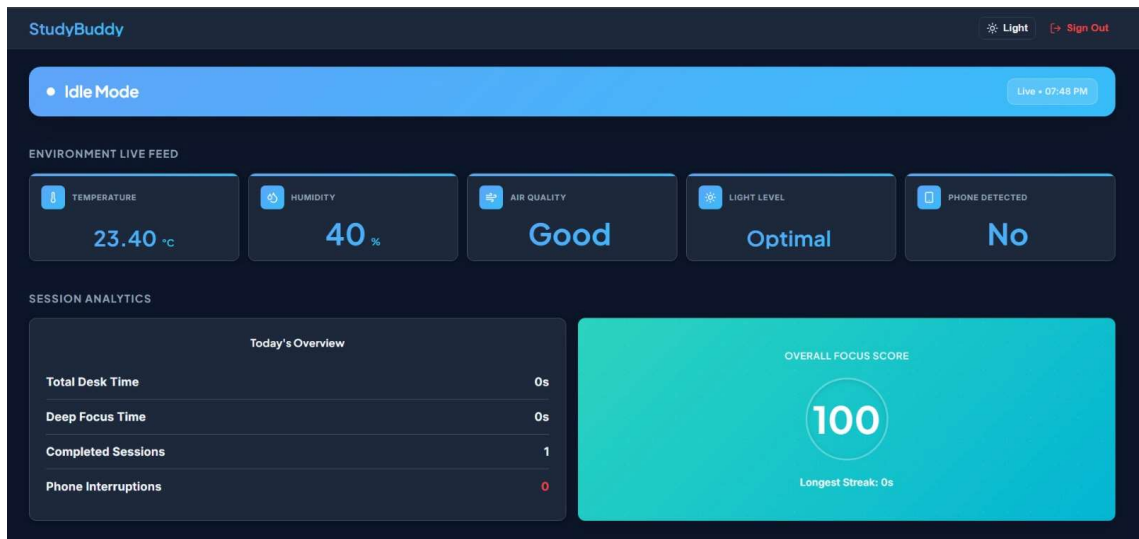


Fig. 3 Night Mode View on the Dashboard

4.4 Hardware Information

- Starting a Session:** This is done by placement of the user's phone in the dedicated compartment. The ultrasonic sensor will detect its presence, automatically switching the website from "Idle" to "Study Mode." The LED strip will also change from blue to green to confirm it is in a study session and the display on the device will showcase the incrementation of the current studying time.

- **During The Session:** As long as the phone remains in the compartment, focus time will continue to accumulate.
- **Phone Removal:** If the phone is removed, without the intention of ending the study session, the LED strip will turn **red**.
- **Break Reminders:** To prevent burnout, Study Buddy will give a notification every 45 minutes that it is time for a short break. The notification can be seen on the display, and the LED strip will turn **purple**.
- **Ending a Session:** When the user is ready to complete a study session, simply pressing the physical button on the device will indicate the device to exit study mode. The final statistics will be saved and updated on the dashboard.

The device is designed to store the information throughout an entire day, and the dashboard statistics are not stored after the device is powered off and disconnected from the website.

5 Conclusions

5.1 Results

The "Study Buddy" project successfully demonstrates the integration of IoT technologies to improve personal productivity and wellbeing. By combining the hardware configuration detailed in Section 3.1 with the multi-layered software architecture described in Section 3.2, the system provides a seamless data pipeline from physical sensors to a cloud-based dashboard.

The final prototype achieves all its core objectives:

- **Reliable Focus Tracking:** The ultrasonic-based phone dock provides an objective measure of "Deep Work" time.
- **Environmental Awareness:** Real-time monitoring of air quality and light levels successfully triggers local visual alerts (LED strip/OLED) and remote updates.
- **Seamless Cloud Sync:** Data is transmitted to Firebase and visualized on a web dashboard with minimal latency, allowing for long-term habit analysis.

The modular nature of the design ensures that the application can be replicated and tested by other users, provided they follow the assembly and calibration instructions in the User Manual (Chapter 4).

5.2 Possibilities for Improvement and Further Development

While the current version of the Study Buddy is a functional and effective tool, several enhancements could be implemented in future iterations to increase its utility:

- **Biometric User Profiles:** Integrating fingerprints or facial recognition would allow the system to recognize different users sharing the same desk. This would enable

personalized study presets (e.g., specific lighting preferences or individual productivity scores).

- **Dedicated Mobile Application:** Developing a native iOS or Android app instead of a browser-based webpage would allow for push notifications. The system could alert the user's phone directly when air quality is poor or when a long study session requires a break.
- **Computer Vision for Posture:** Expanding the "camera" concept to include an integrated webcam using AI. This could monitor the user's spinal alignment and eye-blink rate, providing more advanced physical health feedback than a single ultrasonic sensor.
- **Ecosystem Integration:** Sending data not just to a dashboard, but to external services via the Internet. For example, the system could automatically put the user's phone on "Do Not Disturb" via IFTTT or log study hours directly into a Google Calendar or Notion workspace.
- **Machine Learning Analytics:** Using the stored data to predict when a user is likely to lose focus based on environmental factors (e.g., "Your focus drops by 20% when the room temperature exceeds 24°C"), providing truly personalized coaching.

6 Bibliography

[1] Arduino CC. "Arduino Uno Rev3 Official Product Documentation," [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3> . [Accessed: April 2026].

[2] Espressif Systems. "ESP8266 NodeMCU Series Datasheet," [Online]. Available: <https://www.espressif.com/en/products/socs/esp8266> . [Accessed: April 2026].

[3] Google Firebase. "Firebase Realtime Database Documentation," [Online]. Available: <https://firebase.google.com/docs/database> . [Accessed: May 2026].

[4] Adafruit Industries. "Adafruit NeoPixel Überguide," [Online]. Available: <https://learn.adafruit.com/adafruit-neopixel-uberguide> . [Accessed: March 2026].

[5] Olisci. "MQ-135 Gas Sensor Datasheet for Air Quality Monitoring," Technical Specification, [Online].

[6] Aosong Electronics. "DHT11 Humidity & Temperature Sensor Manual," Technical Specification, [Online].

[7] Oli Kraus. "U8g2 Graphic Library for Monochrome Displays," GitHub Repository, [Online]. Available: <https://github.com/olikraus/u8g2> .

[8] NXP Semiconductors. "I2C-bus specification and user manual," UM10204, [Online].

[9] IETF. "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, [Online].

- [10] Cirillo, F. The Pomodoro Technique: The Acclaimed Time-Management System that Has Transformed How We Work. Currency Publishing, 2018.
- [11] Cook, D. J., & Das, S. K. Smart Environments: Technology, Protocols and Applications. Wiley-Interscience, 2004.
- [12] Ashton, K. "That 'Internet of Things' Thing," RFID Journal, vol. 22, pp. 97-114, 2009. (The foundational text defining IoT).
- [13] <https://www.tinkercad.com/circuits>